Ghost Data Still Exists

Team Playbook: complete task plan for a production-grade submission

HackerRank Orchestrate | October 2026 | Team of two

Team repo: https://github.com/apachedata66/hackoween_hallo.git

Dev tool: Antigravity (must log every turn to log.txt per AGENTS.md)

> **North star.** Two things win this: (1) the engine gets as many of the 518 cases right as possible, because that is what is scored; (2) the tool is genuinely easy for a non-technical person to use, because that is how we stand out in the AI interview and the quality review. **Rule of precedence: engine correctness first, user convenience a very close second, and the UI must never be allowed to change the engine's output.**

How to use this document. Section 1-3 explain the problem and rules. Section 4-6 define architecture and the contract between you two. Section 7-9 are the task lists for Person A and Person B plus joint tasks. Section 10 is the user-convenience spec. Section 11-16 cover quality, testing, evaluation, deliverables, timeline and risks. The appendices have the master prompt and reference tables.

Honest caveat. This plan is built from the participant brief only. We have not yet seen the real dataset, so anything marked **[CONFIRM]** must be checked against dataset/ and the official problem_statement.md before it is treated as fact. If the official repo and this playbook disagree, the official repo wins.

Contents

#	Section
1	The problem in plain English
2	Scoring, priorities and success criteria
3	Output contract (exact rules)
4	System architecture and repository layout
5	Shared data contract between Person A and Person B
6	Working agreement (roles, git, logging)
7	Person A: Data and Evidence Engine (full task list)
8	Person B: Policy, Output and Experience (full task list)
9	Joint tasks
10	User-convenience specification (top priority)
11	Production-grade quality checklist
12	Testing and evaluation plan
13	Deliverables and packaging
14	Timeline and milestones
15	Risks and mitigations
16	Questions to confirm with organisers
17	Final submission checklist
A	Appendix A: Master prompt for Antigravity
B	Appendix B: Reference tables and examples
C	Appendix C: Edge-case catalogue
D	Appendix D: Interview preparation

1. The problem in plain English

A company (Northstar Shop) says a customer's data was deleted. It was not fully deleted: copies remain in secondary databases, support attachments, search indexes, analytics exports, logs, archives and backups, and a sync job even re-created data after deletion. The old workflow's dashboard says "Complete" anyway.

We build one program (the agent) that runs the same logic on every case in deletion_requests.csv (518 cases) and answers:

- Who is this person, including every old and new email or identifier they ever had?
- At the audit time (audit_at), what data of theirs actually still exists, where, and in what form?
- What should have happened to each copy according to deletion_rules.csv (delete, redact, anonymize, or retain)?
- Is anything still able to recreate the data (sync, import, restore from backup)?
- Final verdict: COMPLETE, INCOMPLETE, or NOT_APPLICABLE, plus an ordered list of findings, fixes and checks.

A case is COMPLETE only if all four hold

- Every relevant copy received the treatment that deletion_rules.csv requires.
- No active sync, import or restore path can recreate the data.
- Retained records contain only the identifiers the rules permit.
- Records that must be retained are still available.

> **The golden rule.** The workflow's reported_status and any logged operation result are **claims, not proof**. Only the evidence (SQL snapshots, logs, JSON, PDFs, images, system_activity, audit log) counts. Never copy the dashboard's answer.

Inputs (all inside dataset/)

File	Purpose
deletion_requests.csv	The 518 cases to audit. Fields: case_id, email, requested_at, reported_status, reported_at, audit_at, request_text
sample_deletion_requests.csv / sample_output.csv	Sample cases with correct answers. Use ONLY for format checks and accuracy measurement.
user_identities.csv	Current identities of users
user_audit_log.csv	Historical identity changes (email changes etc.)
system_activity.csv	Deletion, sync, export, backup, restore, indexing and related events
deletion_rules.csv	Required treatment per data category
evidence_files.csv	Manifest of evidence files
evidence/	PDFs, images, logs, JSON, SQLite-compatible SQL snapshot exports (primary, regional, reporting, failover replicas)
output.csv	Blank submission template

2. Scoring, priorities and success criteria

What the hidden grader checks

- Correct deletion_status per case.
- Resolution of current and historical identities.
- Discovery of remaining records and active recreation or restore risks.
- Correct application of deletion policy and evidence references.
- Completeness and ordering of findings, remediation and verification steps.

There is no pass/fail. We get a score against hidden canonical answers. We can only measure ourselves on the sample cases, so building a strong scorer and our own extra test cases is how we find bugs without seeing hidden answers.

Priority ladder (decide trade-offs with this)

Rank	Priority	Why
P0	Engine accuracy and a strictly valid output.csv	This is what is scored. An invalid file can lose everything.
P0	User convenience of the tool (CLI + UI + explanations)	Our chosen differentiator; judged in the AI interview and code review.
P0	All mandatory deliverables present and correct	Missing reports/transcript/zip rules can hurt or disqualify.
P1	Production-grade engineering (tests, logging, config, error handling)	Quality bar we set for ourselves.
P2	Nice-to-haves (per-case PDF reports, dark mode, i18n)	Only after P0 and P1 are solid.

Definition of done for the whole project

- [ ] output.csv: valid, 518 rows, sorted, passes our validator with zero errors.
- [ ] Sample accuracy measured and documented; failures analysed.
- [ ] At least 3 (target 10+) custom test cases with expected outputs.
- [ ] One-command run for CLI and UI on a clean machine.
- [ ] Zero crashes on any case: bad evidence produces a friendly flagged result, never a stack trace.
- [ ] evaluation_report.md and usage_report.md complete and honest.
- [ ] code.zip clean (no dataset, no secrets, no caches, no nested zips).
- [ ] log.txt transcript valid and secret-free.
- [ ] Both teammates can explain every part in the interview.

3. Output contract (exact rules)

output.csv must have the header **case_id,deletion_status,steps_json**, exactly **518 data rows**, sorted by case_id, one row per request.

Allowed phase / operation / state combinations (nothing else is valid)

Phase	Operation	State	Meaning
FINDING	REPORT_REMAINING_RECORD	PRESENT	Record that should be deleted still exists
FINDING	REPORT_EXPOSED_PII	PRESENT	Record may stay but still holds identifiers that should be redacted/anonymized
FINDING	REPORT_ACTIVE_DATA_FLOW	ACTIVE	Active process can copy or recreate the data
FINDING	REPORT_RESTORE_RISK	ACTIVE	Backup or restore path can bring data back
REMEDIATION	DISABLE_DATA_FLOW	DISABLED	Stop the process that copies/recreates data
REMEDIATION	BLOCK_RESTORE	BLOCKED	Prevent affected backup from restoring the user's data
REMEDIATION	DELETE_RECORD	ABSENT	Remove the entire record
REMEDIATION	REDACT_RECORD	REDACTED	Keep record, remove identifying fields required by policy
REMEDIATION	ANONYMIZE_RECORD	ANONYMIZED	Keep record, irreversibly unlink from the user
VERIFICATION	VERIFY_RECORD_ABSENT	ABSENT	Confirm record no longer exists
VERIFICATION	VERIFY_RECORD_PRESENT	PRESENT	Confirm a policy-required retained record still exists
VERIFICATION	VERIFY_RECORD_REDACTED	REDACTED	Confirm required identifying fields are removed
VERIFICATION	VERIFY_RECORD_ANONYMIZED	ANONYMIZED	Confirm record can no longer be linked to the user
VERIFICATION	VERIFY_DATA_FLOW_DISABLED	DISABLED	Confirm the data flow is inactive

Phase	Operation	State	Meaning
VERIFICATION	VERIFY_RESTORE_BLOCKED	BLOCKED	Confirm the data cannot be restored

Step object

```
{"step_no": 1, "phase": "FINDING", "system_id": "account_service", "resource_id": "account_005",
 "operation": "REPORT_REMAINING_RECORD", "state": "PRESENT", "evidence_id": "evidence_01"}
```

Ordering (strict)

- Sort by **phase**: FINDING, then REMEDIATION, then VERIFICATION.
- Within a phase, sort by the **operation sequence** in the table above (top to bottom within that phase).
- Within the same operation, sort by **system_id** ascending, then **resource_id** ascending.
- step_no starts at 1 and increases by 1 with no gaps. JSON key order and whitespace are ignored by the grader; array order is not.
- Confirm [CONFIRM] whether ascending sort is plain string order or natural order (e.g. account_10 vs account_9). Test with the samples.

What each status must contain

Status	steps_json must contain	evidence_id rule
COMPLETE	Verification steps only. They must prove the current state.	Every step cites a real file_id, activity_id or audit_id.
INCOMPLETE	Findings + the complete remediation plan + the verification checks that must pass afterwards.	Findings cite evidence. Proposed remediation and future verification steps use empty string "".
NOT_APPLICABLE	Empty array []	none (no deletion or verification is authorized)

Account tombstones

Some systems keep an account tombstone so permitted records can still reference the same internal user_id. The stable user reference may stay, but direct identifiers must go: name, email, phone and email hash. A tombstone that still holds any of these is exposed PII and needs redaction.

4. System architecture and repository layout

Design principle: **deterministic core, optional LLM at the edges, thin UI on top.** Most of this problem is rule-based (look up, replay, apply rules, sort). Deterministic code is more accurate, reproducible, free and testable. An LLM is only justified where input is unstructured (scanned PDFs, images, free-text request_text), and every LLM call must be cached and logged for the usage report.

```
hackoween_hallo/
  README.md                 one-page quick start + links to docs
  AGENTS.md                 (from organisers) transcript rules - do not edit
  log.txt                   append-only transcript (one machine owns final version)
  pyproject.toml / requirements.txt
  .env.example              variable NAMES only, never values
  .gitignore                dataset/, .env, venv/, __pycache__/, *.zip, cache/
  config/
    settings.yaml           paths, model names, timeouts, feature flags
    prompts/                every LLM prompt as a versioned file
  src/ghostdata/
    contract/               enums, dataclasses, allowed pairs, ordering, validator   (B)
    loaders/                csv, sqlite, log, json, pdf, image loaders               (A)
    identity/               identity resolver                                        (A)
    state/                  state-at-audit_at reconstructor                          (A)
    detectors/              leftover, exposed-PII, data-flow, restore-risk           (A)
    policy/                 rules parser, treatment decision, tombstone logic        (B)
    planner/                status decision, step generator, sorter                  (B)
    explain/                plain-English explanations and timelines                 (B)
    llm/                    client wrapper, cache, retry, token/cost tracker         (shared)
    cli.py                  one-command batch runner                                 (B)
  ui/                       Streamlit (or similar) app - calls src only             (B)
  evaluation/
    score_samples.py        case/step/field/sequence accuracy                        (B)
    custom_cases/           our own cases + expected outputs                         (B+A)
    evaluation_report.md    required deliverable
    usage_report.md         required deliverable
  tests/                    unit + integration + golden tests                        (both)
  docs/                     DATASET_NOTES.md, DECISIONS.md, USER_GUIDE.md, ARCHITECTURE.md
  scripts/                  validate_output.py, make_zip.py, run_full.sh
```

Data flow per case

```
case row -> identity resolver -> all identifiers (current + historical)
         -> evidence loaders (SQL replicas, logs, JSON, PDF/image) -> normalized records
         -> state reconstructor (replay system_activity up to audit_at) -> resource states
         -> detectors (leftovers, exposed PII, active flows, restore risks)
         -> policy engine (required treatment per resource)
         -> status decision -> step generator -> sorter -> validator
         -> output row  (+ explanation object for the UI)
```

Technology choices (confirm in first hour)

Area	Recommendation	Reason
Language	Python 3.11+	Fast to build, SQLite built in, good PDF/OCR libs
Data	csv/pandas (or plain csv), sqlite3 read-only	Snapshots are SQLite-compatible; open read-only so evidence is never modified
PDF/OCR	pdfplumber / pypdf text first, then OCR (pytesseract) or vision LLM fallback	Cheap path first, expensive path only when needed
LLM	Only for OCR/vision and ambiguous free text; provider via env var	Cost, determinism and reproducibility
UI	Streamlit (fastest) or FastAPI + simple HTML	One-command run, minimal build time
Testing	pytest + golden files	Reproducible regression checks
Quality	ruff/black, mypy (optional), pre-commit	Production-grade hygiene

5. Shared data contract between Person A and Person B

This is the most important agreement you two make. **Write it as code in the first hour and commit it before anything else.** A produces it, B consumes it. With this fixed, you can work in parallel and B can build against

hand-written mock data.

```
@dataclass
class ResolvedIdentity:
    case_id: str
    user_ids: list[str]            # internal ids (may be more than one)
    emails_current: list[str]
    emails_historical: list[str]   # from user_audit_log, in order of change
    email_hashes: list[str]        # all hash variants found [CONFIRM hash scheme]
    other_identifiers: dict        # phone, name, etc.
    identity_evidence_ids: list[str]   # audit_id / file_id supporting each link

@dataclass
class ResourceState:
    system_id: str                 # e.g. account_service
    resource_id: str               # e.g. account_005
    kind: str                      # record | tombstone | flow | backup | index | export ...
    data_category: str             # matches a category in deletion_rules.csv
    exists_at_audit: bool
    identifiers_present: list[str] # which direct identifiers still appear: email, phone, name, email_hash ...
    is_active_flow: bool           # for sync/import/export jobs
    can_restore_user_data: bool    # for backups/restore paths
    replica: str | None            # primary | regional | reporting | failover | None
    evidence_id: str               # file_id / activity_id / audit_id that proves this state
    notes: str                     # human-readable reason (feeds the UI explanation)
    confidence: float              # 1.0 for hard evidence; lower if inferred from OCR etc.

@dataclass
class CaseAuditInput:
    request: dict                  # the original row
    identity: ResolvedIdentity
    resources: list[ResourceState]
    warnings: list[str]            # unreadable files, missing evidence, ambiguity
```

> **Rule:** any change to this contract needs both people to agree and a note in docs/DECISIONS.md. Add fields freely; never rename or remove one silently.

6. Working agreement

Roles

	Person A: Data and Evidence Engine	Person B: Policy, Output and Experience
Name	_____	_____
Owns	Everything that reads messy inputs and reconstructs the truth	Everything that turns truth into the answer, and makes it usable
Folders	loaders/, identity/, state/, detectors/	contract/, policy/, planner/, explain/, cli.py, ui/, evaluation/
Produces	CaseAuditInput objects	output.csv, UI, reports, packaging
Backup for	B's explain/ and tests	A's loaders when blocked

Git rules

- main is always runnable. Never push broken code to main.
- Branch per task: feat/a-identity-resolver, feat/b-policy-engine. Small commits, clear messages.
- Pull requests reviewed by the other person (even 5-minute reviews catch interface mistakes). Squash-merge.
- Separate folders mean few conflicts. Touch the other person's folder only via a PR.
- Commit .env.example only. .gitignore must exclude dataset/, .env, caches, venvs and zips BEFORE the first big commit.
- Tag milestones: v0.1 (samples run), v0.5 (full run valid), v1.0 (submission).

Transcript (log.txt) rules, easy to get wrong

- Launch Antigravity from the repo root, or tell it explicitly: read ./AGENTS.md first.
- Each turn logs: User Prompt, Agent Response Summary, Actions, Context. Append only.

- Two laptops means two logs. Agree now which machine produces the final log.txt, or ask organisers whether a combined log is accepted [CONFIRM].
- Never paste keys, tokens, dataset personal data, or credentials into any prompt. Review log.txt before submitting.

Daily rhythm

- 10-minute sync at start and end of each work block: what is done, what is blocked, any interface change.
- Keep a shared checklist (GitHub issues or a TODO.md) mirroring the task IDs in this document.
- Run the sample scorer before every merge to main. Accuracy must never silently drop.

7. Person A: Data and Evidence Engine

Mission: for any case, produce a complete and **evidence-backed** picture of what exists for that person at audit_at. Priority tags: P0 must, P1 should, P2 nice.

ID	Task	Details and acceptance criteria	Pri
A1	Dataset reconnaissance	Profile every file: columns, types, row counts, nulls, duplicates, date formats and timezones, id formats. Open every SQL snapshot and list tables/columns/replica labels. Catalogue evidence types and counts. Write docs/DATASET_NOTES.md. **Acceptance:** both teammates can answer 'what is in each file' without opening it.	P0
A2	Config and typed loaders	Central settings; loaders for each CSV with explicit types; robust datetime parsing (normalize to UTC, decide treatment of naive timestamps [CONFIRM]); strip/normalize whitespace and case. **Acceptance:** loaders raise clear, handled errors, never raw tracebacks.	P0
A3	Identity resolver	From a case email: find user_id(s) in user_identities.csv; walk user_audit_log to find all historical emails (chains a to b to c, changes before and after requested_at); normalize case, whitespace, plus-addressing [CONFIRM if relevant]; find email-hash variants by reverse-engineering the hash used in the data; handle one email mapping to several users and one user with several emails; **never** attach another person's records because an old email was later reused. Emit ResolvedIdentity with evidence ids. **Acceptance:** unit tests for chains, reuse, case differences, unknown email.	P0
A4	SQL snapshot loader	Open SQLite-compatible exports read-only; handle dumps that must be loaded into in-memory DB; enumerate tables; identify which system and replica (primary, regional, reporting, failover) each snapshot represents; handle schema differences between replicas; map snapshot file_id from evidence_files.csv. **Acceptance:** one function returns all rows matching a set of identifiers across ALL snapshots and replicas, tagged with system_id, replica, file_id.	P0
A5	Log and JSON parsers	Parse logs and JSON evidence into normalized events; extract identifiers, timestamps, resource ids; tolerate malformed lines. **Acceptance:** parse errors are counted and surfaced as warnings, not crashes.	P0
A6	PDF and image extraction	Order: embedded text first, OCR second, vision LLM third (only if needed). Cache by file hash. Record extraction method and confidence. Search extracted text for all identifiers (including fuzzy variants like spaced or masked emails [CONFIRM]). **Acceptance:** deterministic results on rerun (cache), cost tracked, low-confidence results flagged in warnings.	P1
A7	State-at-audit_at reconstructor	Replay system_activity (and audit log) in time order up to and including audit_at; ignore events after audit_at; correct tie-breaking for equal timestamps [CONFIRM]; treat logged 'success' as a claim and cross-check with snapshots; detect deleted-then-recreated records (sync or restore after deletion); detect records never deleted despite 'Complete'. **Acceptance:** per-resource timeline that explains how the current state came to be.	P0
A8	Active data-flow detector	Find sync, import, export, indexing jobs that are enabled at audit_at and can copy or recreate the user's data; determine direction and target; ignore flows that were disabled before audit_at. **Acceptance:** each active flow has a system_id, resource_id and evidence_id.	P0
A9	Restore-risk detector	Find backups or restore paths containing the user's data that can restore it; check whether a restore block is actually in effect at audit_at; distinguish 'backup exists' from 'backup can restore into live systems' per the evidence. **Acceptance:** per-backup finding with evidence.	P0
A10	Exposed-PII detector	For records that policy says must be retained, check which direct identifiers (name, email, phone, email hash, others) remain beyond what the rules permit; include tombstones. **Acceptance:** list of identifiers per resource in ResourceState.identifiers_present.	P0
A11	Replica and secondary-system sweep	Systematically check every system and replica, not just account_service; a primary can be clean while a regional or failover copy still holds data. **Acceptance:** coverage report per case listing every system and replica searched, including those with no hits.	P0
A12	Evidence-ID mapping	Every ResourceState carries the correct file_id, activity_id or audit_id; verify each exists in the manifests; choose the most specific supporting evidence. **Acceptance:** validator check that every cited id exists.	P0
A13	Resource model emission	Assemble CaseAuditInput exactly per section 5, with warnings and confidence. **Acceptance:** serializable to JSON for debugging and for UI.	P0

ID	Task	Details and acceptance criteria	Pri
A14	Ambiguity and failure handling	Missing files, corrupt SQLite, unreadable PDFs, conflicting evidence, duplicate ids: log, add warning, continue. Never crash a batch because of one case. **Acceptance:** chaos tests (delete/corrupt a fixture file) produce warnings, not exceptions.	P0
A15	Performance	Index evidence once (by identifier), not per case; parallelize safely; cache extraction. **Acceptance:** full 518-case run completes in a documented, reasonable time; second run is much faster.	P1
A16	Debug tooling	Script: 'explain case X' printing identities, resources, timeline and raw evidence snippets. Essential for error analysis with B. **Acceptance:** any teammate can diagnose a wrong case in minutes.	P1
A17	Unit and integration tests	Test each loader, resolver and detector with small fixtures (tiny fake SQLite, fake logs). **Acceptance:** pytest green; coverage of resolver and reconstructor is high.	P0
A18	LLM cost hooks	Wrap any LLM use in the shared llm/ client so tokens and calls are logged for usage_report.md. **Acceptance:** totals match provider dashboards within a small margin.	P0

8. Person B: Policy, Output and Experience

Mission: turn A's evidence into the exact, valid answer; prove it with a scorer; and make the whole thing pleasant for a non-technical user.

ID	Task	Details and acceptance criteria	Pri
B1	Output contract module	Enums for phase/operation/state; table of allowed combinations; ordering keys; Step and CaseResult dataclasses; JSON serializer; CSV writer with correct quoting (steps_json contains commas and quotes). **Acceptance:** impossible to construct an invalid pair; round-trip test with sample_output.csv.	P0
B2	Strict validator (scripts /validate_output.py)	Check header text and column order, exactly 518 rows, sorted case_id, one row per request, JSON parses to array, step_no consecutive from 1, allowed pairs, correct ordering, status-specific rules (COMPLETE verification-only with real evidence ids; INCOMPLETE proposed steps use empty evidence_id; NOT_APPLICABLE equals []), no duplicate steps. Clear, human-readable error messages. **Acceptance:** catches deliberately broken files in tests.	P0
B3	Policy engine	Parse deletion_rules.csv into a typed rulebook [CONFIRM columns]; map each data_category to required treatment (delete, redact, anonymize, retain) and permitted identifiers; tombstone rule (keep user_id, strip name, email, phone, email hash); retention rule (must-retain stays present). Unknown categories raise a warning, not a guess. **Acceptance:** table-driven tests per rule.	P0
B4	Status decision	Combine detector output + policy: COMPLETE only if all four conditions hold; NOT_APPLICABLE when no deletion or verification is authorized [CONFIRM exact trigger by studying samples and request_text]; otherwise INCOMPLETE. Write the decision as small, named, testable rules, not one giant function. **Acceptance:** each condition has a unit test.	P0
B5	Step generator	INCOMPLETE: findings for every problem, remediation for every finding in the right operation (flow gets DISABLE_DATA_FLOW, backup gets BLOCK_RESTORE, record gets DELETE/REDACT/ANONYMIZE per policy), and a verification step for every remediation plus verification of retained records. COMPLETE: verification steps only with evidence ids. NOT_APPLICABLE: []. **Acceptance:** remediation-to-verification pairing is complete (no orphan steps).	P0
B6	Sorter and numbering	Implement the exact ordering rules; assign step_no last. **Acceptance:** property test: shuffling input never changes output.	P0
B7	Scorer (evaluation/score_samples.py)	Compute case-level status accuracy, step-level accuracy (precision/recall/F1 on step sets), field-level accuracy (per field: system_id, resource_id, operation, state, evidence_id), and sequence accuracy (order). Print a confusion matrix of statuses and a per-case diff for failures. **Acceptance:** reproduces numbers deterministically; used by CI.	P0
B8	Custom evaluation cases	Create 10+ hand-built cases (minimum required: 3) with tiny fixtures and expected outputs: replica-only leftover; sync job recreating data; tombstone with leftover email hash; must-retain record; historical-email case; NOT_APPLICABLE case; reported COMPLETE but actually INCOMPLETE; truly COMPLETE with evidence; restore risk with and without block; email reused by another user. **Acceptance:** all expected outputs reviewed by A.	P0

ID	Task	Details and acceptance criteria	Pri
B9	CLI	One command: ghostdata run --input dataset/deletion_requests.csv --out output.csv. Options: --case ID, --samples, --explain, --no-llm, --resume, --workers. Progress bar, summary table (counts per status, warnings, runtime, LLM cost). Exit codes meaningful. **Acceptance:** runs from a clean clone following README only.	P0
B10	Plain-English explainer	For every case produce: one-sentence verdict, bulleted reasons in everyday language, a timeline of events, and a diff between reported_status and actual status. Template-based (deterministic), not LLM. **Acceptance:** a non-technical reviewer understands the verdict without seeing JSON.	P0
B11	Web UI	See section 10 for the full spec. **Acceptance:** usability checklist in section 10 passes.	P0
B12	Usage and cost tracker	Central llm/ client: records provider, model, calls, input/output tokens, cost per call; aggregates per model and overall; writes usage_report.md for the final full run automatically. **Acceptance:** report matches the run that produced output.csv exactly.	P0
B13	Evaluation report generator	Script that fills evaluation_report.md metrics from the scorer; add narrative on failures found and fixes. **Acceptance:** numbers in the report equal the scorer's output.	P0
B14	README and user guide	README: install, one-command run, CLI and UI usage, config, folder map, troubleshooting. docs/USER_GUIDE.md with screenshots for the UI. **Acceptance:** a teammate who has never seen the repo runs it in under 5 minutes.	P0
B15	Packaging	scripts/make_zip.py builds code.zip excluding dataset/, venvs, caches, nested archives, .env, log.txt (submitted separately); checks size and scans for secrets. **Acceptance:** zip inspected automatically before release.	P0
B16	Per-case report export	Optional HTML or PDF report per case from the explainer. **Acceptance:** opens offline, no external assets.	P2
B17	CI	GitHub Actions: lint, tests, sample scorer, validator on every PR. Block merge if accuracy drops. **Acceptance:** green pipeline on main.	P1

9. Joint tasks

ID	Task	Owner	Notes
J1	Agree and commit the data contract (section 5) and repo skeleton	A + B	First hour. No other coding before this.
J2	Read official problem_statement.md and AGENTS.md together; list ambiguities	A + B	Write answers in docs/DECISIONS.md with the evidence for each decision.
J3	Integration: wire A's output into B's planner; run samples end to end	A + B	Pair on the first integration, do not do it alone.
J4	Error-analysis loop	A + B	For each failing sample case: classify cause (identity, evidence, state replay, policy, ordering, evidence id) and fix the right layer.
J5	Rule discovery from samples	A + B	Study sample_output.csv to infer conventions (which resource gets which operation, id formats, evidence choices). Infer rules, never copy answers.
J6	Security and privacy review	A + B	No secrets in code, config, prompts, transcript. Datasets stay out of git and zip.
J7	Final full run	B drives, A reviews	Freeze code, run once, validate, archive usage report from that exact run.
J8	Transcript hygiene	Whoever owns final log.txt	Check four sections per turn, append-only, no secrets.
J9	Interview rehearsal	A + B	Appendix D. Each person must be able to explain the other's half.
J10	Submission dry run	A + B	Do the final checklist (section 17) 24 hours before the deadline, not at the deadline.

10. User-convenience specification (top priority)

Compliance and deletion-audit tools are usually hostile: raw JSON, cryptic ids, no explanation, no guidance on what to do next. Our target user is a **non-technical support or compliance person** who has two minutes and needs to know: *Was this person's data really deleted? If not, what do I do?*

> **Design commitments.** (1) The answer comes first, jargon later. (2) Every claim links to its evidence. (3) Nothing crashes; everything explains itself. (4) One command to run. (5) The UI is a view on the engine: it can never change a verdict or the CSV.

10.1 Screens and behaviours

Screen / feature	What the user sees	Acceptance
Home / case finder	Search box for case_id or email; recent cases; button to upload a requests CSV; sample data toggle.	Finding a case takes under 10 seconds; typos in email still help (suggestions).
Verdict banner	Big colour-coded badge: green 'Fully deleted - proven', red 'Not fully deleted - N problems', grey 'Nothing to delete'. Never colour alone: also icon and text.	Understandable at a glance; passes colour-blind check.
Plain-English summary	2-4 sentences: what we found and why, e.g. 'Email still exists in the support attachments replica, and a sync job keeps copying it back.'	No internal ids in the first paragraph.
Reported vs actual	Side-by-side: what the old dashboard said vs what the evidence shows, highlighting mismatches.	Mismatch is obvious without reading details.
Identity card	Who this person is: current email, every past email with dates, internal user ids, with evidence links.	Explains why records under an old email were included.
Timeline	Chronological list: requested, deletion run, sync, backup, restore... up to audit_at, with icons; events after audit_at greyed out or hidden.	Shows exactly how the data survived or came back.
Problems list	Grouped by type (leftover records, exposed personal info, active flows, restore risks), each with where, why, and evidence link.	Each item has a one-line 'what this means'.
Fix checklist	Ordered remediation steps as a checklist, with the matching 'how to confirm' step beside each. Copy button, print view.	Matches the official ordering; remediation and verification are paired.
Evidence viewer	Click an evidence id to see the source snippet (log line, table row, PDF page image) with the matching identifier highlighted.	No need to open files manually.
Technical details (collapsed)	Exact steps_json, raw resource states, warnings, confidence. Hidden by default.	Power users can expand; others never see it.
Bulk mode	Upload or select the full file, see a progress bar with ETA, a summary dashboard (counts per status, top problem types), filter and sort the results table, download output.csv.	Download is byte-identical to the CLI output.
Warnings centre	Friendly list: 'This PDF could not be read; result may be incomplete' with suggested action.	No stack traces ever shown.
Export	Download output.csv, per-case report (P2), copy-as-text summary.	One click.

10.2 Usability requirements

- [ ] Time to first result under 5 minutes from a clean clone (README only).
- [ ] Runs with one command per interface (CLI and UI); no manual DB setup.
- [ ] Every error message says what went wrong, why it matters and what to do next, in plain language.
- [ ] Loading states and progress for anything over 1 second; cancel button for bulk runs.
- [ ] Keyboard accessible, readable contrast, text not colour-only, sensible font sizes (basic WCAG-aware design).

[ ] Works on a laptop and a phone-width window.

[ ] Consistent vocabulary: pick one term per concept (e.g. 'leftover copy', 'exposed personal info') and use it everywhere in the UI, the CLI and the docs.

[ ] Glossary tooltips for unavoidable terms (tombstone, replica, anonymize vs redact).

[ ] Safe by default: read-only evidence, no destructive buttons, clear 'simulation only' label (we recommend actions, we do not execute them).

[ ] Privacy-aware: the UI masks emails partially by default with a 'reveal' toggle; nothing is sent to third parties; no telemetry.

[ ] Reproducible: showing the engine version, dataset fingerprint and run time on every result.

10.3 CLI convenience

- --help is genuinely helpful, with examples.
- Human-friendly summary at the end of a run plus a machine-readable JSON summary option.
- --resume continues a long run after interruption.
- --explain CASE_ID prints the same plain-English explanation as the UI.
- Validation runs automatically after every run and prints a pass/fail with fixes.

10.4 Guardrail: convenience must not hurt scoring

> The UI and explainer are read-only views of the engine's CaseResult. One code path produces output.csv. A test must assert that CLI output, UI download and the scorer all use the same function. If time is short, cut UI polish, never engine correctness.

10.5 Usability testing (cheap but real)

- Ask one person outside the team (a junior, a friend) to find 'why case X is incomplete' using only the UI. Watch silently; note every hesitation.
- Fix the top 3 friction points, then retest with a second person.
- Record the findings and fixes in evaluation_report.md; it is great interview material.

11. Production-grade quality checklist

Correctness and determinism

[ ] Same input always gives byte-identical output.csv (seed everything, sort everything, no wall-clock in outputs).
[ ] LLM calls cached on content hash; --no-llm mode works and is the fallback.
[ ] No hardcoded case ids or sample answers anywhere (grep check in CI).
[ ] Timezone and date-parse policy documented and tested.

Robustness

[ ] One bad case never aborts the batch; failures are logged and the case gets a safe, flagged result.
[ ] All file access read-only; evidence never modified.
[ ] Timeouts and retries with backoff on LLM calls; budget cap to prevent runaway cost.
[ ] Handles empty, missing and malformed inputs with explicit tests.

Observability

[ ] Structured logging with levels; per-case debug trace on demand.
[ ] Run manifest: code version (git hash), config, dataset fingerprint, start/end time, counts, warnings.
[ ] Warnings aggregated in the run summary.

Security and privacy

[ ] Secrets only in environment variables; .env.example lists names only.
[ ] Pre-commit and zip-time secret scan (keys, tokens, emails from dataset).
[ ] Dataset excluded from git and zip; no personal data in logs beyond what is necessary (mask emails in logs).
[ ] Dependency list pinned; minimal dependencies.

Code quality

[ ] Type hints on public functions; docstrings on modules.
[ ] Small, single-purpose modules; policy rules data-driven, not buried in if-chains.
[ ] Linting and formatting enforced; no dead code or commented-out blocks.
[ ] docs/DECISIONS.md records every non-obvious choice with the reason and evidence.

Documentation

[ ] README quick start, USER_GUIDE for non-technical users, ARCHITECTURE for engineers, DATASET_NOTES for the data.
[ ] Every CLI flag and config option documented.
[ ] Troubleshooting section with the five most likely problems.

12. Testing and evaluation plan

12.1 Test layers

Layer	What	Owner
Unit	Parsers, identity chains, rule lookup, sorter, validator, each detector	A and B
Property	Sorting is order-independent; step_no always consecutive; validator rejects any invalid pair	B
Golden	Fixed fixtures with expected CaseResult JSON, reviewed by both	A + B
Integration	Full pipeline on sample cases; output equals expectations where known	A + B
Chaos	Corrupt or missing evidence; empty files; duplicate ids; huge file	A
Regression	Every bug found gets a permanent test	Both
Usability	Outsider test of the UI (section 10.5)	B

12.2 Metrics to report (required in evaluation_report.md)

- **Case-level status accuracy** with confusion matrix across COMPLETE / INCOMPLETE / NOT_APPLICABLE.
- **Step-level accuracy**: precision, recall, F1 over (phase, system_id, resource_id, operation, state) tuples.
- **Field-level accuracy** per field, including evidence_id correctness.
- **Sequence accuracy**: ordering correctness (e.g. normalized edit distance or exact-order match rate).
- **Per-stage diagnostics**: identity-resolution accuracy, leftover recall, active-flow recall, restore-risk recall (measure what you can from samples).
- At least 3 custom cases (target 10+) with description, expected result, actual result.
- Failures found and improvements made, as a dated list with before/after numbers.

12.3 Custom case ideas (build as many as time allows)

#	Scenario	Expected
1	Primary deleted, regional replica still holds record	INCOMPLETE: remaining record finding + delete + verify absent
2	Sync job re-created the deleted record after deletion	INCOMPLETE: active flow + remaining record; disable flow before delete
3	Tombstone keeps email hash and phone	INCOMPLETE: exposed PII + redact + verify redacted
4	Record required to be retained, identifiers beyond permitted	INCOMPLETE: exposed PII; redact/anonymize; verify present and redacted
5	Fully clean case, all evidence present	COMPLETE: verification steps only with real evidence ids
6	Dashboard says COMPLETE but backup can restore data	INCOMPLETE: restore risk + block restore + verify blocked
7	Dashboard says NOT_STARTED but data already gone	COMPLETE (evidence proves it) - tests we do not trust the claim either way
8	User changed email twice; data under oldest email	Identity chain resolved; leftover found under historical email
9	Old email later reused by a different user	Other user's records NOT touched
10	No deletion authorized or applicable	NOT_APPLICABLE and []
11	Event happens just after audit_at	Ignored for state at audit_at
12	Unreadable evidence PDF	Warning surfaced; result flagged, no crash

12.4 Error-analysis loop (repeat until diminishing returns)

- Run scorer, list failing sample cases, sort by frequency of cause.
- Diagnose with scripts/explain: was the identity wrong, evidence missed, state replay wrong, policy wrong, ordering wrong or evidence id wrong?
- Fix at the right layer, add a regression test, rerun, record the before/after metric.
- Beware overfitting to few samples: prefer fixes that follow the written rules over fixes that merely match one sample.

13. Deliverables and packaging

Deliverable	Contents	Owner
output.csv	518 rows + header, sorted, valid, from the final full run	B (A reviews)
code.zip	Runnable code, prompts, config, README.md, evaluation/ folder. NO dataset, venv, caches, nested archives, secrets	B
evaluation/evaluation_report.md	Case-level accuracy, step and field accuracy, sequence accuracy, 3+ custom cases, failures and improvements	B + A
evaluation/usage_report.md	For the FINAL full run: providers and models, number of calls, input/output/total tokens, average tokens per request, estimated total and per-request cost, per-model and overall totals	B

Deliverable	Contents	Owner
log.txt (chat_transcript)	Antigravity transcript with User Prompt, Agent Response Summary, Actions, Context per turn; append-only; no secrets	Designated owner
AI interview	Explain approach, architecture, evaluation, and the user-convenience layer	Both

Usage report reminders

- If we use no LLM for the final run, say so explicitly and report zero tokens and zero cost. Honest zero is better than invented numbers.
- If LLMs are used for OCR/vision, report exactly those calls. Numbers must come from the tracker, not memory.
- Re-run the full dataset only once from frozen code; the report must describe that run.

14. Timeline and milestones

Fill in real dates once the deadline is confirmed. Work backwards from the deadline and leave at least a day of buffer.

Phase	Goal	Person A	Person B	Exit criteria
0. Setup (hour 0-2)	Repo, tools, contract	Dataset recon (A1)	Skeleton, .gitignore, contract (B1)	Contract committed; AGENTS.md logging works
1. Foundations	Loaders, contract, validator	A2-A5	B1, B2, B3	Samples parse; validator passes on sample_output.csv
2. First end-to-end	Samples run to output	A3, A7, A11, A13	B4, B5, B6, B7	Scorer prints first accuracy number
3. Accuracy push	Error-analysis loop	A8-A10, A12, A14	B8, scorer diffs	Sample accuracy plateau; regression tests
4. Experience	CLI + UI + explainer	A16 debug tooling, evidence viewer data	B9, B10, B11	Usability checklist passes; outsider test done
5. Hardening	Quality, performance, docs	A15, A17	B12-B14, B17	CI green; clean-clone run works
6. Full run and package	Freeze, run, report, zip	Review	J7, B13, B15	All deliverables ready 24h before deadline
7. Dry run and interview prep	Submit rehearsal	Both	Both	Checklist (section 17) all ticked

> **Cut order if time runs short** (cut from the bottom): per-case PDF export, dark mode, bulk-mode extras, UI polish beyond the checklist, vision-LLM fallback, performance tuning. **Never cut:** validator, scorer, evidence ids, reports, transcript, packaging rules.

15. Risks and mitigations

Risk	Impact	Mitigation
Misreading ordering rules	Large loss on every INCOMPLETE case	Validator + property tests; check natural vs string sort against samples
Trusting reported status	Wrong status on many cases	Design rule: never read reported_status when deciding; use it only for the UI diff
Missing replica or secondary system	Missed leftovers	Coverage report per case listing all systems and replicas searched
Historical identity gaps	Missed records under old emails	Resolver tests; trace chains; check hashes of old emails
Over-matching (email reuse by another user)	Deleting someone else's data	Tie records to user_id, not just email; test case 9
Wrong evidence ids on COMPLETE	Lost points; unverifiable proof	Validator checks ids exist; prefer most specific evidence

Risk	Impact	Mitigation
Overfitting to samples	Poor hidden score	Rule-based design; custom cases; avoid per-sample hacks
LLM nondeterminism and cost	Unreproducible output, budget blowout	Deterministic core, cache, --no-llm fallback, budget cap
Transcript missing or malformed	Submission item invalid	Launch from repo root, check each turn, one owner for log.txt
Secrets or dataset in zip/transcript	Disqualification risk	Secret scan, .gitignore first, review before upload
Merge conflicts / interface drift	Lost time	Separate folders, contract file, PR reviews, short syncs
UI eats time	Engine accuracy suffers	Engine first; UI built only on the stable CaseResult; cut order in section 14
Late discovery of format issues	Last-minute chaos	Validate output from the very first end-to-end run, not at the end

16. Questions to confirm with organisers

- [ ] Are API keys or credits provided for LLM use, or do we use our own? Any restrictions on models or providers for the final run?
- [ ] Is a deterministic non-LLM solution acceptable (with LLM only for OCR)? [Expected yes, confirm]
- [ ] Which log.txt counts for a two-person team using two machines? Is a combined log accepted?
- [ ] Exact deadline, upload portal and file size limits for code.zip and log.txt.
- [ ] Is sorting by system_id and resource_id plain string order or natural order?
- [ ] How is NOT_APPLICABLE determined (what in request_text or the data triggers it)?
- [ ] Timezone handling for audit_at and activity timestamps when no zone is given.
- [ ] Format and schedule of the AI interview; are both members expected to attend?
- [ ] Is our team repo (hackoween_hallo) fine as the working repo, with only the zip submitted?
- [ ] Any rule about a UI or extra tooling beyond the required outputs (to be safe, confirm it does not violate the rules).

17. Final submission checklist

- [ ] output.csv header is exactly: case_id,deletion_status,steps_json.
- [ ] Exactly 518 data rows, sorted by case_id, one per request.
- [ ] Every steps_json parses as a JSON array; step numbers consecutive from 1.
- [ ] Every operation/state pair is allowed; steps ordered by phase, operation, system_id, resource_id.
- [ ] COMPLETE cases: verification steps only, each with a real evidence id.
- [ ] INCOMPLETE cases: findings + complete remediation + future verification; proposed steps use evidence_id "".
- [ ] NOT_APPLICABLE cases: [] exactly.
- [ ] code.zip includes runnable code, prompts, config, README.md, evaluation/evaluation_report.md, evaluation/usage_report.md.
- [ ] code.zip excludes dataset, venv, caches, nested archives, .env.
- [ ] usage_report.md describes the final full-dataset run that produced output.csv, with numbers taken from the tracker.
- [ ] evaluation_report.md lists case, step, field and sequence accuracy, 3+ custom cases, failures and improvements.
- [ ] log.txt follows the four-section turn format, append-only, contains no secrets.
- [ ] No keys, tokens, personal data or credentials anywhere in the submission.
- [ ] Clean-clone test: a fresh checkout runs CLI and UI following README only.
- [ ] Both teammates rehearsed the interview and can explain every module.

[ ] Submitted at least a few hours before the deadline.

Appendix A: Master prompt for Antigravity

Paste this as the first message. It already includes the correction that hackoween_hallo is OUR team workspace and the official files come from the organisers' repo.

```
MASTER PROMPT: HackerRank Orchestrate (Oct 2026), "Ghost Data Still Exists"

ROLE: Lead engineer for a 2-person team. Build a deletion-audit agent AND a user-friendly interface.
Show a plan before coding. Ask about ambiguities instead of guessing.

SETUP
- Team repo: https://github.com/apachedata66/hackoween_hallo.git (OUR shared workspace, not the official repo)
- Official challenge files (dataset/, AGENTS.md, problem_statement.md) come from
  https://github.com/interviewstreet/hackerrank-orchestrate-october26 . If missing here, tell me what to copy.
- Read ./AGENTS.md and problem_statement.md fully before coding. Official files win over any summary.
- Follow AGENTS.md: append an entry to log.txt after EVERY turn with sections
  User Prompt, Agent Response Summary, Actions, Context. Append only.
- Two-person workflow: feature branches, small commits, folders core/ ui/ evaluation/ kept clean.
  .gitignore for dataset/, venvs, caches, .env, zips.
- Never put API keys, tokens or private data in code, config, prompts or log.txt. Use env vars.

PROBLEM
For each of 518 cases in dataset/deletion_requests.csv determine whether the person's data was really deleted.
reported_status and operation results are CLAIMS, not proof. Resolve current + historical identifiers, reconst
evidence (all SQL replicas, logs, JSON, PDFs, images), find leftovers, exposed PII, active data flows, restore
deletion_rules.csv (delete/redact/anonymize/retain, incl. account tombstones: keep user_id, remove name/email/
COMPLETE only if: all copies treated per rules; no sync/import/restore can recreate data; retained records hol
identifiers; must-retain records remain available.

OUTPUT CONTRACT (enforce with a validator)
output.csv columns exactly case_id,deletion_status,steps_json; 518 rows sorted by case_id.
Status in {COMPLETE, INCOMPLETE, NOT_APPLICABLE}. Step = {step_no, phase, system_id, resource_id, operation, s
Allowed pairs only (see the table in the brief). Order: phase, operation sequence, system_id asc, resource_id
COMPLETE = verification steps only with real evidence ids. INCOMPLETE = findings + full remediation + verifica
evidence_id "" on proposed steps. NOT_APPLICABLE = [].

ARCHITECTURE: deterministic core (identity, loaders, state reconstructor, detectors, policy, planner, validato
OCR/vision/ambiguous text with caching and token logging; CLI; UI that only calls core and never changes outpu

USABILITY (top priority): the target user is non-technical. Build a one-command local web UI with: case search
verdict (green/red/grey with icon+text), reasons in everyday language, reported-vs-actual comparison, identity
timeline, grouped problems, ordered fix checklist paired with verification, evidence viewer, collapsed technic
with progress and download identical to CLI output, friendly errors (never stack traces), read-only/simulation
with reveal toggle, accessible design. Engine accuracy comes first; UI must not change engine output.

QUALITY: deterministic outputs, tests (unit, property, golden, chaos), structured logging, run manifest, CI, d
(README, USER_GUIDE, ARCHITECTURE, DATASET_NOTES, DECISIONS), secret scan, packaging script.

FAIRNESS: never hardcode sample answers; never seek hidden answers; use samples only for format and accuracy m

DELIVERABLES: output.csv, code.zip (no dataset), evaluation/evaluation_report.md (case/step/field/sequence acc
failures and fixes), evaluation/usage_report.md (final full run: models, calls, tokens, averages, cost), log.t

WORK ORDER: 1) read files + explore dataset; 2) plan + ambiguity list, then STOP for approval; 3) contract + v
4) core engine to sample accuracy; 5) CLI; 6) UI; 7) reports, README, packaging, final validation.
```

Appendix B: Reference tables and examples

Example: INCOMPLETE case (illustrative only, not real data)

```
[
 {"step_no":1,"phase":"FINDING","system_id":"account_service","resource_id":"account_005",
  "operation":"REPORT_REMAINING_RECORD","state":"PRESENT","evidence_id":"evidence_01"},
 {"step_no":2,"phase":"FINDING","system_id":"sync_job","resource_id":"sync_002",
  "operation":"REPORT_ACTIVE_DATA_FLOW","state":"ACTIVE","evidence_id":"activity_17"},
 {"step_no":3,"phase":"REMEDIATION","system_id":"sync_job","resource_id":"sync_002",
  "operation":"DISABLE_DATA_FLOW","state":"DISABLED","evidence_id":""},
 {"step_no":4,"phase":"REMEDIATION","system_id":"account_service","resource_id":"account_005",
  "operation":"DELETE_RECORD","state":"ABSENT","evidence_id":""},
 {"step_no":5,"phase":"VERIFICATION","system_id":"account_service","resource_id":"account_005",
  "operation":"VERIFY_RECORD_ABSENT","state":"ABSENT","evidence_id":""},
 {"step_no":6,"phase":"VERIFICATION","system_id":"sync_job","resource_id":"sync_002",
  "operation":"VERIFY_DATA_FLOW_DISABLED","state":"DISABLED","evidence_id":""}
```

```
]
```

Notice the pattern: every finding has a matching remediation and a matching verification. Check sample_output.csv for the real conventions [CONFIRM].

Finding to remediation to verification map (starting hypothesis, verify against samples)

Finding	Remediation	Verification
REPORT_REMAINING_RECORD	DELETE_RECORD (or ANONYMIZE_RECORD if rules say so)	VERIFY_RECORD_ABSENT (or VERIFY_RECORD_ANONYMIZED)
REPORT_EXPOSED_PII	REDACT_RECORD or ANONYMIZE_RECORD per rules	VERIFY_RECORD_REDACTED or VERIFY_RECORD_ANONYMIZED
REPORT_ACTIVE_DATA_FLOW	DISABLE_DATA_FLOW	VERIFY_DATA_FLOW_DISABLED
REPORT_RESTORE_RISK	BLOCK_RESTORE	VERIFY_RESTORE_BLOCKED
(retained record by policy)	none needed if already correct	VERIFY_RECORD_PRESENT

Suggested decision pseudocode (status)

```
if no deletion or verification authorized:            status = NOT_APPLICABLE ; steps = []
else:
    problems = leftovers + exposed_pii + active_flows + restore_risks + retained_missing
    if problems is empty:                              status = COMPLETE   ; steps = verification-only w/ evid
    else:                                              status = INCOMPLETE ; steps = findings + remediation + v
```

Appendix C: Edge-case catalogue

- Email case differences, leading/trailing spaces, plus-addressing, unicode lookalikes.
- Email changed multiple times; change dated before or after requested_at.
- Same email belonged to two users at different times.
- Multiple user_ids for one person (duplicate accounts).
- Event timestamps exactly equal to audit_at or requested_at; events in different timezones.
- Deletion logged as success but record still present in a replica.
- Deletion logged as failed but record actually absent.
- Record deleted, then re-created by sync, then not deleted again.
- Backup taken before deletion; restore path active vs blocked.
- Restore event after deletion that re-introduced data.
- Tombstone present with user_id but also email hash or phone.
- Record category that policy says to retain but with extra identifiers.
- Category not found in deletion_rules.csv.
- Resource id formats that sort differently as strings vs numbers.
- Evidence file listed in manifest but missing on disk.
- Corrupt or empty SQLite snapshot; snapshot with different schema per replica.
- PDF that is a scan with no text layer; image with partially masked email.
- Very large log file.
- Case with request_text that conflicts with the data (customer says X, evidence says Y).
- Duplicate case rows or repeated emails across cases.

Appendix D: Interview preparation

Both of you should be able to answer all of these, including about the other person's half.

- What is the problem in one minute, and why is a deletion dashboard not proof?
- Walk through one case from email to final steps_json.
- Why a deterministic core with LLM only at the edges? What did the LLM add and what did it cost (cite the usage report)?

- How do you resolve historical identities, and how do you avoid touching another user's data?
- How do you decide COMPLETE vs INCOMPLETE vs NOT_APPLICABLE?
- How do you handle replicas, tombstones, active flows and restore risks?
- How did you evaluate, what were your worst failures, and what changed afterwards?
- How do you guarantee valid, reproducible output (validator, determinism)?
- What did you do to make it usable for a non-technical person, and how did you test that?
- What would you do with more time (scale, real-system integration, safe execution of remediation)?
- What are the limitations and risks of your approach, honestly?
- How did you split work and keep integration safe?

> **Final reminder.** Engine accuracy and valid output first. Evidence, not claims. Plain-English answers for humans. Keep secrets and the dataset out of git, the zip and the transcript. Dry-run the submission a day early.